

7.3 子集生成

第7.2节中介绍了排列生成算法。本节介绍子集生成算法：给定一个集合，枚举所有可能的子集。为了简单起见，本节讨论的集合中没有重复元素。

7.3.1 增量构造法

第一种思路是一次选出一个元素放到集合中，程序如下：

```
void print_subset(int n, int* A, int cur) {
    for(int i = 0; i < cur; i++) printf("%d ", A[i]);    //打印当前集合
    printf("\n");
    int s = cur ? A[cur-1]+1 : 0;                        //确定当前元素的最小可能值
    for(int i = s; i < n; i++) {
        A[cur] = i;
        print_subset(n, A, cur+1);                      //递归构造子集
    }
}
```

和前面不同，由于 A 中的元素个数不确定，每次递归调用都要输出当前集合。另外，递归边界也不需要显式确定——如果无法继续添加元素，自然就不会再递归了。

上面的代码用到了定序的技巧：规定集合 A 中所有元素的编号从小到大排列，就不会把集合 $\{1, 2\}$ 按照 $\{1, 2\}$ 和 $\{2, 1\}$ 输出两次了。

提示 7-4：在枚举子集的增量法中，需要使用定序的技巧，避免同一个集合枚举两次。

这棵解答树上有 1024 个结点。这不难理解：每个可能的 A 都对应一个结点，而 n 元素集合恰好有 2^n 个子集， $2^{10}=1024$ 。

7.3.2 位向量法

第二种思路是构造一个位向量 $B[i]$ ，而不是直接构造子集 A 本身，其中 $B[i]=1$ ，当且仅当 i 在子集 A 中。递归实现如下：

```
void print_subset(int n, int* B, int cur) {
    if(cur == n) {
        for(int i = 0; i < cur; i++)
            if(B[i]) printf("%d ", i);                //打印当前集合
        printf("\n");
        return;
    }
    B[cur] = 1;                                        //选第 cur 个元素
```

```

print_subset(n, B, cur+1);
B[cur] = 0;
print_subset(n, B, cur+1);
}
//不选第 cur 个元素

```

必须当“所有元素是否选择”全部确定完毕后才是一个完整的子集，因此仍然像以前那样当 $\text{if}(\text{cur} = n)$ 成立时才输出。现在的解答树上有 2047 个结点，比刚才的方法略多。这个也不难理解：所有部分解（不完整的解）也对应着解答树上的结点。

提示 7-5：在枚举子集的位向量法中，解答树的结点数略多，但在多数情况下仍然够快。

这是一棵 $n+1$ 层的二叉树（ cur 的范围从 $0 \sim n$ ），第 0 层有 1 个结点，第 1 层有 2 个结点，第 2 层有 4 个结点，第 3 层有 8 个结点，……，第 i 层有 2^i 个结点，总数为 $1+2+4+8+\dots+2^n=2^{n+1}-1$ ，和实验结果一致。如图 7-2 所示为这棵解答树。

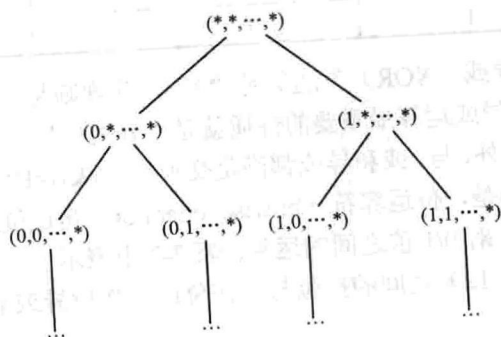


图 7-2 位向量法的解答树

这棵树依然符合前面的观察结果：最后几层结点数占整棵树的绝大多数。

7.3.3 二进制法

另外，还可以用二进制来表示 $\{0, 1, 2, \dots, n-1\}$ 的子集 S ：从右往左第 i 位（各位从 0 开始编号）表示元素 i 是否在集合 S 中。图 7-3 展示了二进制 0100011000110111 是如何表示集合 $\{0, 1, 2, 4, 5, 9, 10, 14\}$ 的。

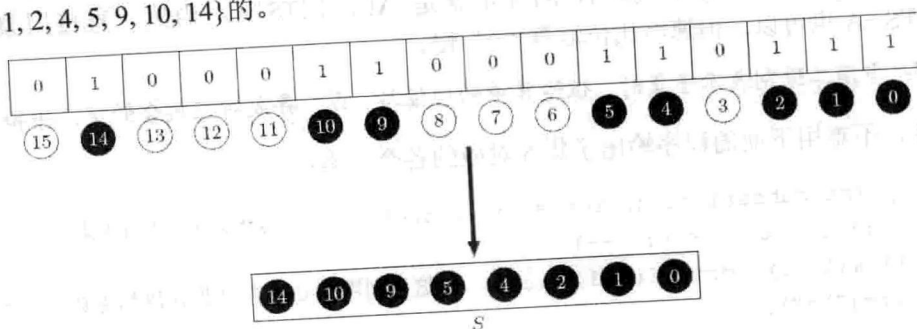


图 7-3 用二进制表示子集

注意：为了处理方便，最右边的位总是对应元素 0，而不是元素 1。

提示 7-6：可以用二进制表示子集，其中从右往左第 i 位（从 0 开始编号）表示元素 i 是否在集合中（1 表示“在”，0 表示“不在”）。

此时仅表示出集合是不够的，还需要对集合进行操作。幸运的是，常见的集合运算都可以用位运算符简单实现。最常见的二元位运算是与（&）、或（|）、非（!），它们和对应的逻辑运算非常相似，如表 7-1 所示。

表 7-1 C 语言中的二元位运算

A	B	A & B	A B	A ^ B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

表 7-1 中包括了“异或（XOR）”运算符“^”，其规则是“如果 A 和 B 不相同，则 $A \wedge B$ 为 1，否则为 0”。异或运算最重要的性质就是“开关性”——异或两次以后相当于没有异或，即 $A \wedge B \wedge B = A$ 。另外，与、或和异或都满足交换律： $A \& B = B \& A$ ， $A | B = B | A$ ， $A \wedge B = B \wedge A$ 。

与逻辑运算符不同的是，位运算符（bitwise operator）是逐位进行的——两个 32 位整数的“按位与”相当于 32 对 0/1 值之间的运算。表 7-2 中表示了二进制数 10110（十进制为 22）和 01100（十进制为 12）之间的按位与、按位或、按位异或的值，以及对应的集合运算的含义。

表 7-2 位运算与集合运算

	A	B	A & B	A B	A ^ B
二进制	10110	01100	00100	11110	11010
集合	{1,2,4}	{2,3}	{2}	{1,2,3,4}	{1,3,4}

不难看出， $A \& B$ 、 $A | B$ 和 $A \wedge B$ 分别对应集合的交、并和对称差。另外，空集为 0，全集 $\{0, 1, 2, \dots, n-1\}$ 的二进制为 n 个 1，即十进制的 $2^n - 1$ 。为了方便，往往在程序中把全集定义为 $\text{ALL_BITS} = (1 \ll n) - 1$ ，则 A 的补集就是 $\text{ALL_BITS} \wedge A$ 。当然，直接用整数减法 $\text{ALL_BITS} - A$ 也可以，但速度比位运算“^”慢。

提示 7-7：当用二进制表示子集时，位运算中的按位与、或、异或对应集合的交、并和对称差。

这样，不难用下面的程序输出子集 S 对应的各个元素：

```
void print_subset(int n, int s) { //打印{0, 1, 2, ..., n-1}的子集 S
    for(int i = 0; i < n; i++)
        if(s & (1 << i)) printf("%d ", i); //这里利用了 C 语言“非 0 值都为真”的规定
    printf("\n");
}
```